باسمه تعالی

آزمایشگاه طراحی سیستم‌های دیجیتال

دانشکده مهندسی کامپیوتر دانشگاه صنعتی شریف

تابستان ۱۴۰۳

استاد:

دکتر محسن انصاری

مهندس محمدرضا حسینی

اعضای گروه:

محمدامین عباس‌فر – ۴۰۱۱۰۶۲۱۱

سعید فراتی کاشانی – ۴۰۱۱۰۶۲۹۹

امیرحسین صوری – ۴۰۱۱۰۶۱۸۲

فهرست عناوین

موضوع آزمایش ۳
شرح کلی آزمایش ۳
پیاده‌سازی ۴
واحد کنترل ۴
ورودی‌ها ۵
خروجی‌ها ۵
توضیحات ۵
مسیر داده ۶
ورودی‌ها ۶
خروجی‌ها ۷
توضیحات ۷
ماژول نهایی ۸
ورودی‌ها ۸
خروجی‌ها ۹
توضیحات ۹
برنامه‌ی محک (TestBench) ۹
تست اول ۹
تست دوم ۱۰
تست سوم ۱۰

موضوع آزمایش

طراحی ضرب کننده

شرح کلی آزمایش

۲-۶ آزمایش پنجم: طراحی ضرب کننده

هدف از این آزمایش طراحی یک واحد ضرب کننده است که برای انجام عمل ضرب از روش ضرب بوث (Booth) استفاده می‌کند. برای انجام این آزمایش مسیرداده و واحد کنترل را جداگانه طراحی کنید و سپس با اتصال آن‌ها به یکدیگر ضرب کننده را ایجاد کنید.

توجه کنید که در این طراحی هنگامی که مضروب فیه (Multiplier) را به سمت راست شیفت می‌دهید لازم است که واحد شیفت دهنده توان انجام شیفت بیش از یک بیت در یک پالس ساعت را داشته باشد تا بتواند نسبت به ضرب عادی (Shift & ADD) تسریع داشته باشد. لذا حتماً در طراحی خود از چنین واحد شیفت دهنده‌ای استفاده کنید.

می دانیم که برای پیاده سازی الگوریتم booth برای ضرب، کافی است دو رجیستر n بیتی و یک dff داشته باشیم. در ابتدا در رجیستر اول و dff مقدار صفر و در رجیستر دوم مقدار عملوند اول را قرار میدهیم. مانند شکل زیر:

00000 11011 0

در هر مرحله اگر lsb رجیستر دوم و dff مقادیر زیر را داشت، عملیات مربوطه را انجام میدهیم (در مثال بالا کافی است دو بیت lsb را در نظر بگیریم):

00 → no arithmetic operation

01 → add multiplicand to left half of product

10 → subtract multiplicand from left half of product

11 → no arithmetic operation

بعد از هر مرحله هم یک arithmetic right shift به روی هر سه رجیستر با هم انجام میدهیم. بعد از n مرحله انجام این کار، نتیجه نهایی در رجیستر اول و دوم بدست می آید.

پیاده‌سازی

ابتدا دو قسمت اصلی controller که وظیفه تعیین سیگنال های کنترلی و datapath که مسیر داده است را پیاده سازی می کنیم و با اتصال این دو، الگوریتم booth را تکمیل می نماییم.

واحد کنترل

```
module controller
(
    input start,
    input [1:0] lsb_bits,
    input clk,
    output ready,
    output [1:0] operation,     // 0 : nothing, 1: add, 2: sub
    output init_signal
);

    assign operation = lsb_bits[0] == lsb_bits[1] ? 2'b00 : (lsb_bits[1] ? 2'b10 : 2'b01);
    assign ready = counter == 0;
    assign init_signal = counter == 0;

    reg [3:0] counter = 0;
    always @(posedge clk) begin
        if((start & ready) | (~ready)) begin
            if(counter == 4'b1000)
                counter <= 0;
            else
                counter <= counter + 1;
        end
    end
```

ورودی‌ها

start: سیگنال start مدار برای شروع فرایند ضرب

Lsb_bits: دو بیت کم ارزش که در توضیحات کلی توضیح داده شد.

Clk: سیگنال کلاک

خروجی‌ها

Ready: سیگنالی که به کاربر پایان فرایند ضرب را اعلام مینماید

Operation: سیگنالی که به واحد محاسبات میگوید کدام یک از عملیات جمع، تفریق و یا هیچ کدام را انجام دهد.

Init_signal: سیگنالی که مشخص میکند رجیستر ها از مقدار اولیه load کنند (برای رجیستر های اول و dff این مقدار برابر صفر و برای رجیستر دوم برابر عملوند اول می باشد)

توضیحات

با توجه به دو بیت lsb_bits و جدولی که در توضیحات کلی مربوط به عملیات جمع و تفریق آمده، متغیر operation مقدار دهی می شود. همچنین در این ماژول که شمارنده داریم که صفر بودن آن به معنای حالت ready و شروع نشدن فرایند ضرب است. با زدن دکمه start این شمارنده تا عدد ۸ زیاد می شود (در اینجا n را برابر ۸ در نظر گرفته ایم) و با انجام عملیات محاسباتی برای بار ۸ ام مجددا به صفر برمیگردد و در وضعیت ready قرار میگیرد. با توجه به این نکات، متغیر های ready, init_signal مقدار دهی شده اند. در بلوک always هم کار های مربوط به زیاد شدن و reset شدن شمارنده انجام گرفته است.

مسیر داده

```
module datapath(
    input [7:0] operand1,
    input [7:0] operand2,
    input [1:0] operation,
    input init_signal,
    input clk,
    output reg [16:0] calculating_bits
);

    wire [16:0] alu_result;
    alu alu_ins(
        .calculating_bits(calculating_bits),
        .operand2(operand2),
        .operation(operation),
        .result(alu_result)
    );

    always @(posedge clk) begin
        if(init_signal) begin
            calculating_bits[0] <= 0;
            calculating_bits[8:1] <= operand1;
            calculating_bits[16:9] <= 0;
        end
        else begin
            calculating_bits <= alu_result;
        end
    end

endmodule
```

ورودی‌ها

Operand۱: عملوند اول ضرب

Operand۲: عملوند دوم ضرب

Operation: عملیاتی که روی رجیستر اول باید انجام شود که توسط واحد کنترل مشخص میگردد.

Init_signal: سیگنالی که مقدار دهی اولیه رجیستر ها را مشخص میکند

Clk: سیگنال کلاک

خروجی‌ها

Calculating_bits: یک رجیستر ۱۷ بیتی که از چپ به راست، ۸ بیت اول آن برای رجیستر اول، ۸ بیت دوم آن برای رجیستر دوم و بیت آخر آن برای dff می باشد.

توضیحات

در مسیر داده با توجه به سیگنال operation حاصل عملیات محاسباتی روی رجیستر اول را محاسبه میکنیم. این کار را با ماژول alu انجام داده ایم که تصویر آن در زیر مشاهده می شود:

```
module alu(
    input [16:0] calculating_bits,
    input [7:0] operand2,
    input [1:0] operation,
    output [16:0] result
);
    wire [16:0] add_res = {calculating_bits[16:9] + operand2, calculating_bits[8:0]};
    wire [16:0] sub_res = {calculating_bits[16:9] - operand2, calculating_bits[8:0]};
    wire [16:0] nop_res = calculating_bits;

    wire [16:0] mux_res = operation == 2'b00 ? nop_res : (operation == 2'b01 ? add_res : sub_res);

    // arithmetic shift right
    assign result = {mux_res[16], mux_res[16:1]};

endmodule
```

در این ماژول صرفا با توجه به operation جمع و یا تفریق و یا هیچ کدام را انجام می دهیم و سپس کل حاصل را arithmetic right shift می دهیم.

همچنین در مسیر داده در هر کلاک، اگر واحد کنترل سیگنال init_signal را فعال کرده باشد، مقادیر اولیه را درون رجیستر ها لود میکند و در غیر این صورت حاصل alu را درون رجیستر ها میریزد.

ماژول نهایی

```
module booth_multiplier
(
    input [7:0] operand1,
    input [7:0] operand2,
    input start,
    input clk,
    output ready,
    output reg [15:0] result
);
    wire [16:0] calculating_bits;
    wire [1:0] operation;
    wire init_signal;

    controller control(
        .start(start),
        .lsb_bits(calculating_bits[1:0]),
        .clk(clk),
        .ready(ready),
        .operation(operation),
        .init_signal(init_signal)
    );

    datapath path(
        .operand1(operand1),
        .operand2(operand2),
        .operation(operation),
        .init_signal(init_signal),
        .clk(clk),
        .calculating_bits(calculating_bits)
    );

    always @(posedge ready) begin
        result <= calculating_bits[16:1];
    end

endmodule
```

ورودی‌ها

Operand۱: عملوند اول ضرب

Operand۲: عملوند دوم ضرب

Start: سیگنال شروع فرایند ضرب

Clk: سیگنال کلاک

خروجی‌ها

Ready: سیگنالی که به کاربر پایان فرایند ضرب و آمادگی برای انجام ضرب بعدی را اعلام میکند.

Result: خروجی حاصل ضرب که همزمان با فعال شدن سیگنال ready مقدار میگیرد.

توضیحات

در این ماژول دو واحد کنترل و مسیر داده را به هم متصل کرده ایم. همچنین از آنجا که خروجی از طریق این ماژول به کاربر داده می شود، با فعال شدن سیگنال ready و بدست آمدن حاصل ضرب، مقدار حاصل ضرب که در رجیستر های اول و دوم موجود است را به خروجی result منتقل میکنیم.

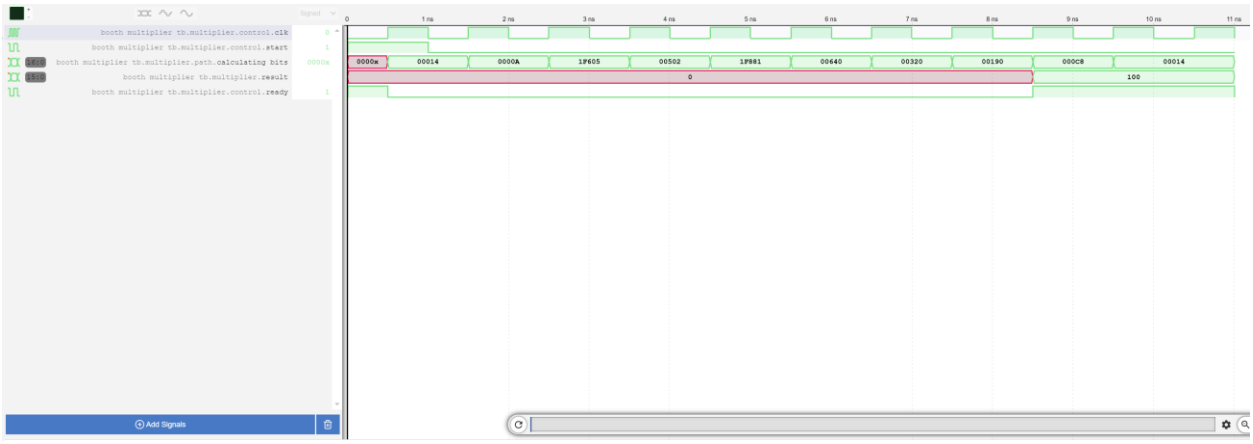
برنامه‌ی محک (TestBench)

در این بخش ۴ تست نوشته شده برای این برنامه را با waveform های مربوطه بررسی میکنیم.

تست اول

```
$display("test 1: ");
A = 10;
B = 10;
start = 1; #1 start = 0;
```

```
test 1:
time =    11, A = 10, B = 10, output = 100, ready = 1,
booth_multiplier_tb.v:59: $finish called at 110 (100ps)
```

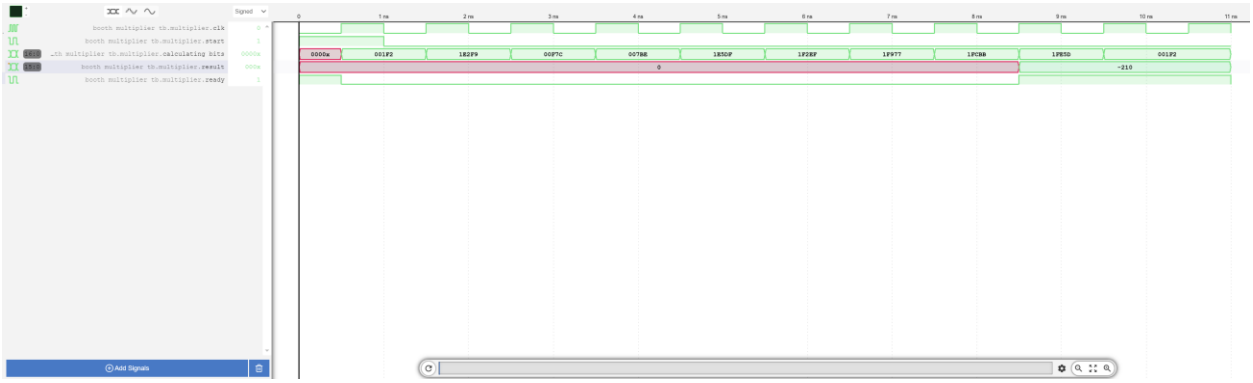


همانطور که از نتیجه پیداست، حاصل ۱۰۰ = ۱۰ * ۱۰ به درستی محاسبه شده است.

تست دوم

```
$display("test 2: ");
A = -7;
B = 30;
start = 1; #1 start = 0;
```

```
test 2:
time =    11, A = -7, B = 30, output = -210, ready = 1,
booth_multiplier_tb.v:59: $finish called at 110 (100ps)
```

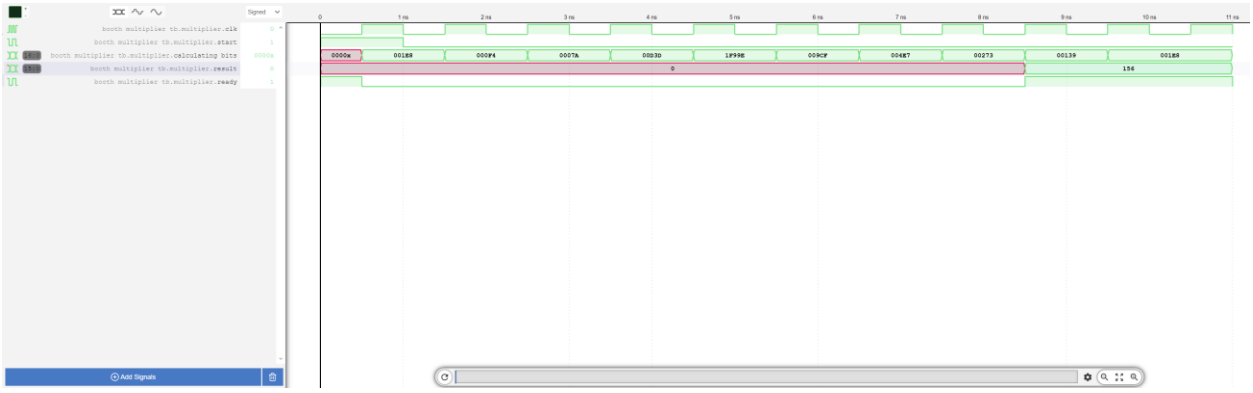


همانطور که از نتیجه پیداست، حاصل ۲۱۰− = ۳۰ * ۷− به درستی محاسبه شده است.

تست سوم

```
$display("test 3: ");
A = -12;
B = -13;
start = 1; #1 start = 0;
```

```
test 3:
time =    11, A = -12, B = -13, output = 156, ready = 1,
booth_multiplier_tb.v:52: $finish called at 110 (100ps)
```



همانطور که از نتیجه پیداست، حاصل ۱۵۶ = (۱۳−) * ۱۲− به درستی محاسبه شده است.